Projeto Final: Planejamento Financeiro

Do Orçamento Pessoal à Modelagem Computacional em Python

Prof. Reginaldo Fernandes
Lógica de Programação com Python

Instituto Federal do Ceará – Campus Tauá
Curso Técnico em Redes de Computadores

14 de outubro de 2026

Roteiro da Apresentação

1. O Desafio do Planejamento Financeiro
2. Arquitetura e Fluxo do Sistema
3. Modelagem dos Dados em Python
4. Lógica de Negócio e Diagnóstico
5. Nossa Jornada: As 5 Aulas do Projeto

O Problema: Para Onde Vai o Dinheiro?

Dificuldades comuns no dia a dia:

- Falta de visibilidade clara entre o que entra e o que sai.
- Gastos invisíveis (pequenas compras rotineiras que acumulam).
- Ausência de reserva para imprevistos e emergências.
- Sensação de que o dinheiro acaba antes do fim do mês.

O Papel da Computação

Softwares orçamentários transformam números brutos em **métricas visuais** e **diagnósticos automáticos**, auxiliando na tomada de decisão consciente.

A Metodologia Orçamentária: Regra 50-30-20

A regra divide a receita líquida mensal em três categorias estratégicas:

50% Essenciais

Necessidades vitais:

- Moradia (aluguel)
- Alimentação
- Transporte
- Contas de consumo
- Saúde

30% Estilo de Vida

Desejos pessoais:

- Lazer e passeios
- Assinaturas digitais
- Restaurantes
- Compras e hobbies

20% Reserva e Metas

Segurança e futuro:

- Reserva de emergência
- Quitação de dívidas
- Investimentos
- Projetos de longo prazo

A Visão Geral da Solução em Python

O sistema foi concebido como uma aplicação CLI (Interface em Linha de Comando) dividida em 3 pilares fundamentais:

1. Entrada e Estado

- Previsão de Renda.
- Lançamento seguro de Despesas.
- Validação de tipos com `try/except`.

2. Estrutura

- Dicionários (`dict`) para cada gasto.
- Lista (`list`) agregadora.
- Menus em ciclo (`while True`).

3. Inteligência

- Somatórios por categoria.
- Cálculo de saldo real.
- Avaliação percentual da regra 50-30-20.

O Ciclo de Vida do Software (Listening Loop)

Como o programa se mantém vivo no terminal?

- **Inicialização:** Configura o estado inicial na memória (`renda = 0.0`, `despesas = []`).
- **Laço Principal:** O comando `while True` garante que o menu não se encerre após uma ação.
- **Escuta:** O comando `input()` aguarda a decisão do usuário técnico.
- **Desvio Condicional:** `if / elif / else` encaminha para a função especialista responsável.
- **Término Gracioso:** A opção `"0"` executa o comando `break`, liberando o terminal.

Modelagem: Dicionário como Registro Individual

Analogia de Redes: Assim como uma entrada na tabela ARP contém IP, MAC e Interface, cada despesa no software possui campos bem definidos:

```python
# Cada gasto do mundo real torna-se um Dicionario
despesa = {
    "descricao": "Aluguel da Residencia",
    "valor": 1200.00,
    "categoria": "Essencial"
}

# Acesso direto e legivel atraves das chaves:
print(despesa["descricao"]) # Imprime: Aluguel da Residencia
print(despesa["valor"])     # Imprime: 1200.0
```

Vantagem do Dicionário

Substitui variáveis soltas e desorganizadas por um **objeto estruturado** em que cada dado possui significado explícito por chave textual.

Modelagem: Lista como Coleção de Registros

Para armazenar múltiplos lançamentos ao longo do mês, agrupamos os dicionários dentro de uma lista dinâmica:

```
# Colecao inicial vazia
despesas = []

# Novos registros sao inseridos com .append()
despesas.append({
    "descricao": "Internet Fibra",
    "valor": 110.0,
    "categoria": "Essencial"
})
despesas.append({
    "descricao": "Cinema e Pipoca",
    "valor": 65.0,
    "categoria": "Estilo de Vida"
})

# Quantidade total armazenada:
print(len(despesas)) # Retorna: 2
```

O Motor de Análise: Acumuladores e Diagnóstico

Como o algoritmo processa os dados na Opção 4?

```
# 1. Acumuladores condicionais com laco for:
for item in despesas:
    if item["categoria"] == "Essencial":
        total_essencial += item["valor"]
    elif item["categoria"] == "Estilo de Vida":
        total_estilo_vida += item["valor"]
    elif item["categoria"] == "Reserva":
        total_reserva += item["valor"]

# 2. Calculo do saldo liquido:
total_gasto = total_essencial + total_estilo_vida + total_reserva
saldo_restante = renda_prevista - total_gasto

# 3. Diagnostico percentual (Meta 50-30-20):
if renda_prevista > 0:
    pct_essencial = (total_essencial / renda_prevista) * 100
```

Diagnóstico Automatizado da Saúde Financeira

O sistema classifica a situação financeira em três estados orçamentários:

- **Orçamento Deficitário (`saldo < 0`):** Os gastos superam as entradas. Alerta vermelho acionado para cortes imediatos.
- **Orçamento no Limite (`saldo == 0`):** Saldo zerado. Não há reserva para contingências.
- **Orçamento Superavitário (`saldo > 0`):** Sobra financeira. Recursos disponíveis para poupança e metas futuras.

Defesa Contra Divisão por Zero

A verificação `if renda_prevista > 0:` impede que o Python quebre com `ZeroDivisionError` caso o usuário acesse o relatório sem registrar a receita.

Mapa de Construção Passo a Passo

Construiremos o software juntos, aula a aula, em pequenas fatias funcionais:

1. **Aula 1:** Estrutura do Programa, Loop Principal e Menu Interativo.
2. **Aula 2:** Validação de Entrada com `try/except` e Previsão de Renda.
3. **Aula 3:** Cadastro de Despesas com Listas e Dicionários.
4. **Aula 4:** Listagem Formatada em Tabela e UX no Terminal.
5. **Aula 5:** Inteligência de Negócio, Regra 50-30-20 e Entrega Final.

"Não tente escrever 200 linhas de uma vez. Programadores profissionais constroem software um bloco funcional por vez!"

Mãos à Obra!

Vamos iniciar com a **Etapa 1**: o menu e o fluxo de controle.

Instituto Federal do Ceará – Campus Tauá
Curso Técnico em Redes de Computadores